Score Contributor 3: Unnecessary Repetition

SAME STEP AS THE OMISSION METRIC!!

To do this, we first try to obtain the whole list of elements that must be animated throughout the whole process, irrespective of the actual sequence. We can't get this from the XML, because not all elements in the XML may explicitly make it to the actual animated list.

We must use the Animation Sequence instead

We take the Animation Sequence, the original diagram code, and the diagram image and form an exhaustive list of elements that must get animated in any animation video. We pass the code so that we can ground these elements from their IDs to their actual textual labels.

We must first use this prompt

Element Checklist Obtaining Script

```
You are an expert Diagram Code Analyst and Visual Evaluator.
Your objective is to process an animation sequence JSON, cros
s-reference it with the original diagram code and original im
age, and extract a comprehensive, human-readable checklist of
every structural element (nodes, blocks, edges) that makes an
appearance in the animation.

You will be provided with:
```

```
1. The Original Image of the diagram.
2. The Original Diagram Code (which contains the raw definitions and text labels).
3. The Animation Sequence JSON (which lists elements by their IDs across various timestamps).

*** EXTRACTION RULES ***

1. Node and Block Naming: To name a node or block, trace its ID from the animation JSON back to the original code and extract the text label directly associated with it.
2. Unlabeled Blocks/Rasters: If a block, raster image, or node is completely unlabeled in the code, look at the Original Image and assign it a concise, semantically meaningful name (e.g., "Blue Convolution Box", "Input Raster Image").
3. Composite Elements: For composite elements that contain sub-parts, you must only include the textual label of the *whole* parent element in the checklist. Do not clutter the checklist with the names of internal sub-components.
4. Edge/Arrow Formatting: For arrows, extract the resolved semantic names of both the source and target elements. Format the output strictly as: `from: "[Source Name]" to: "[Target Name]"`.
5. Exclude Standalone Text: Do not create a checklist category for raw textual labels. We only care if the structural components (blocks, nodes, arrows) are influenced by the style. Use the text labels *only* to name the blocks and nodes they belong to.

*** TASK ***
Iterate through EVERY timestamp in the Animation Sequence JSON. Accumulate all the resolved, human-readable labels for `nodes`, `blocks`, and `arrows`. Deduplicate the lists so each element only appears once.

*** OUTPUT REQUIREMENT ***
```

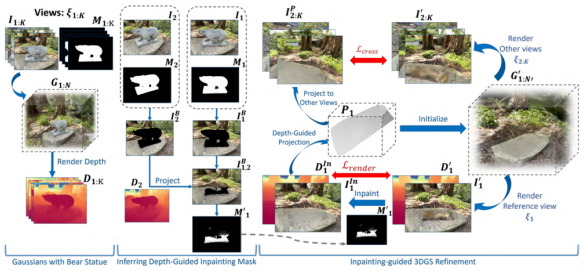
```
Output ONLY valid JSON matching the exact schema below. Do not output conversational text.

### EXAMPLE LOGIC ###
If the JSON has a block `"G_box"`, and the code shows this box has the text label "Generator".
If the JSON has an arrow from `"I_stack_1"` (labeled "Input Images") to `"G_box"`.
Your output arrays will store: "Generator" in blocks, and "from: 'Input Images' to: 'Generator'" in edges.

```json
{
  "animation_checklist": {
    "blocks": [
      "Generator",
      "Projection Box P1",
      "Feature Extractor (Unlabeled Blue Box)"
    ],
    "nodes": [
      "Input Image Stack",
      "Depth Map D1",
      "Mask M1"
    ],
    "edges": [
      "from: 'Input Image Stack' to: 'Generator'",
      "from: 'Projection Box P1' to: 'Feature Extractor (Unlabeled Blue Box)'"
    ]
  }
}
```

An example element checklist

```
{
  "animation_checklist": {
```
```




```
    "blocks": [
      "Gaussian Block $G_
{1:N}$",
      "Box I2, M2",
      "Box I1, M1",
      "Projection Box $P_1
$",
      "Gaussian Block $G'_
{1:N'}$"
    ],
    "nodes": [
      "Image Stack ($I_{1:
K}$)",
      "Mask Stack ($M_{1:
K}$)",
      "Depth Stack ($D_{1:
K}$)",
      "Image $I_2$",
      "Mask $M_2$",
      "Image $I_1$",
      "Mask $M_1$",
      "Masked $I_2$",
      "Masked $I_1$",
      "Depth $D_2$",
      "Projected $I_2$",
      "Inpaint Mask $M'_1
$",
      "Depth $D'_1$",
      "Rendered $I'_1$",
      "Composite Depth/Ima
ge $D'_1, I'_1$",
      "Small Mask $M'_1$",
      "Depth $D^{In}_1$",
      "Inpainted $I^{In}_1
$",
      "Composite Depth/Ima
```

```
ge $D^{In}_1, I^{In}_1$",
      "Projection Stack $I
^P_{2:K}$",
      "Render Stack $I'_
{2:K}$"
    ],
    "edges": [
      "from: 'Image Stack
($I_{1:K}$)' to: 'Gaussian
Block $G_{1:N}$'",
      "from: 'Gaussian Blo
ck $G_{1:N}$' to: 'Depth S
tack ($D_{1:K}$)'",
      "from: 'Mask $M_2$'
to: 'Masked $I_2$'",
      "from: 'Mask $M_1$'
to: 'Masked $I_1$'",
      "from: 'Masked $I_2
$' to: 'Depth $D_2$'",
      "from: 'Masked $I_1
$' to: 'Projected $I_2$'",
      "from: 'Depth $D_2$'
to: 'Projected $I_2$'",
      "from: 'Projected $I
_2$' to: 'Inpaint Mask
$M'_1$'",
      "from: 'Inpaint Mask
$M'_1$' to: 'Small Mask
$M'_1$'",
      "from: 'Rendered
$I'_1$' to: 'Inpainted $I^
{In}_1$'",
      "from: 'Small Mask
$M'_1$' to: 'Inpainted $I^
{In}_1$'",
      "from: 'Depth $D^{I
```

```
n}_1$' to: 'Projection Box
$P_1$'",
      "from: 'Projection B
ox $P_1$' to: 'Projection
Stack $I^P_{2:K}$'",
      "from: 'Projection B
ox $P_1$' to: 'Gaussian Bl
ock $G'_{1:N'}$",
      "from: 'Gaussian Blo
ck $G'_{1:N'}$' to: 'Rende
r Stack $I'_{2:K}$'",
      "from: 'Gaussian Blo
ck $G'_{1:N'}$' to: 'Rende
red $I'_1$'",
      "from: 'Projection S
tack $I^P_{2:K}$' to: 'Ren
der Stack $I'_{2:K}$'",
      "from: 'Depth $D^{I
n}_1$' to: 'Depth $D'_1$'"
    ]
  }
}
```

After this, we use the following prompts, once again with a generic instruction section and a style adaptation section, that help us detect omitted elements while adjusting to the peculiarities of every style

Alpha Masking

```
You are an expert Diagram Animation Repetition Detector. Your
objective is to track the frequency of appearances of element
s from a precomputed checklist and verify if any elements or
arrows are erroneously repeated/re-highlighted in the animati
on sequence.
```

```
You are provided with:
1. The Original Diagram Image.
2. A sequence of Animation Frames in chronological order.
3. The Initial Animation Checklist containing "blocks", "node
s", and "edges".

*** GENERIC INSTRUCTIONS ***
1. Frequency Counter: Treat the checklist as a frequency coun
ter where every block, node, and edge starts at `0`.
2. The State Change Rule (CRITICAL): An element's frequency i
ncreases by +1 ONLY when it undergoes a state change (e.g., f
rom masked to unmasked). Simply persisting in an unmasked sta
te across multiple frames DOES NOT increase the frequency.
3. Repetition Justification: If an element's frequency reache
s 2 or more, you MUST reason if it is justified before flaggi
ng it. Justified repetitions include cyclic edges, overview-f
irst traversals (where a parent container is shown, then mask
ed for children, then unmasked again), or returning to a cent
ral node. If unjustified, flag it.
4. Semantic Mapping: Use the Original Image to logically map
descriptive names in the checklist to visual shapes. Do not r
ely solely on exact text matches.
5. Track All: You must track and update frequencies for block
s, nodes, AND edges.

*** STYLE ADAPTER: ALPHA MASKING ***
In Alpha Masking, active elements fade in to opaque, while in
active elements are masked to a lower opacity.
- State Change Trigger: Increase frequency (+1) the exact mom
ent an element transitions from masked (transparent/low opaci
ty) to fully opaque/active.

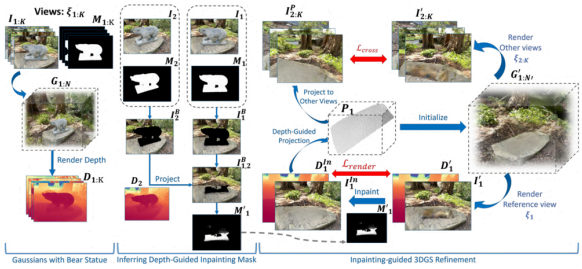
*** OUTPUT REQUIREMENT ***
Output ONLY valid JSON matching the exact schema below.
```

```json
{
  "metadata": {
    "animation_style": "Alpha Masking"
  },
  "timestep_reasoning": [
    {
      "frame_label": "string - frame filename or index",
      "general_frame_reasoning": "Detailed rationale of what
newly became opaque this frame based on the state change rul
e.",
      "frequency_updates_this_frame": {
        "blocks": [{"name": "Block Name", "new_frequency":
1}],
        "nodes": [{"name": "Node Name", "new_frequency": 1}],
        "edges": [{"name": "Edge Name", "new_frequency": 1}]
      },
      "repetition_evaluation": {
        "justification_reasoning": "Deeply reason about ANY e
lement whose frequency is now >= 2. Is this a cyclic edge? A
parent container revisit? Or a glitch? If no elements reached
frequency >= 2, state 'No repetitions to evaluate'.",
        "unjustified_freq_increase_element": ["List of block/
node names whose unjustified frequency increased this fram
e"],
        "unjustified_freq_increase_arrow": ["List of edge nam
es whose unjustified frequency increased this frame"]
      }
    }
  ],
  "final_unjustified_repetitions": {
    "elements": ["Cumulative list of all unjustified repeated
blocks/nodes"],
    "arrows": ["Cumulative list of all unjustified repeated e
dges"]
  },
```

```
  "unnecessary_repetition_count_element": 0,
  "unnecessary_repetition_count_arrow": 0
}
```


We get the following output for AnimateBanana video frames

attachment:85495e10-afbd-44e0-8e03-a7753e0972d7:CVPR_2025_pipe00004_video_tikz_AM.mp4



Original Diagram

Element Checklist


```
{
  "animation_checklist": {
    "blocks": [
      "Gaussian Block $G_{1:N}$",
      "Box I2, M2",
```



```
{
  "metadata": {
    "animation_style": "Alpha Masking"
  },
  "timestep_reasoning": [
    {
      "frame_label": "frame_0.png",
      "general_frame_reasoning": "All diagram elements are currently masked to low opacity; only the caption is visible.",
      "frequency_updates_this_frame": {
        "blocks": [],
        "nodes": [],
        "edges": []
      },
      "repetition_evaluation": {
        "justification_reasoning": "No repetitions to evaluate",
        "unjustified_freq_increase_element": [],
        "unjustified_freq_increase_arrow": []
```

```
      "Box I1, M1",
      "Projection Box $P_1
$",
      "Gaussian Block $G'_
{1:N'}$"
    ],
    "nodes": [
      "Image Stack ($I_{1:
K}$)",
      "Mask Stack ($M_{1:
K}$)",
      "Depth Stack ($D_{1:
K}$)",
      "Image $I_2$",
      "Mask $M_2$",
      "Image $I_1$",
      "Mask $M_1$",
      "Masked $I_2$",
      "Masked $I_1$",
      "Depth $D_2$",
      "Projected $I_2$",
      "Inpaint Mask $M'_1
$",
      "Depth $D'_1$",
      "Rendered $I'_1$",
      "Composite Depth/Ima
ge $D'_1, I'_1$",
      "Small Mask $M'_1$",
      "Depth $D^{In}_1$",
      "Inpainted $I^{In}_1
$",
      "Composite Depth/Ima
ge $D^{In}_1, I^{In}_1$",
      "Projection Stack $I
^P_{2:K}$",
      "Render Stack $I'_
```

```
      }
    },
    {
      "frame_label": "fram
e_1.png",
      "general_frame_reaso
ning": "The input data sta
cks and the initial 3D mod
el become opaque, along wi
th the edge connecting the
m.",
      "frequency_updates_t
his_frame": {
        "blocks": [
          {
            "name": "Gauss
ian Block $G_{1:N}$",
            "new_frequenc
y": 1
          }
        ],
        "nodes": [
          {
            "name": "Image
Stack ($I_{1:K}$)",
            "new_frequenc
y": 1
          },
          {
            "name": "Mask
Stack ($M_{1:K}$)",
            "new_frequenc
y": 1
          }
        ],
        "edges": [
```

```
{2:K}$"
    ],
    "edges": [
      "from: 'Image Stack
($I_{1:K}$)' to: 'Gaussian
Block $G_{1:N}$'",
      "from: 'Gaussian Blo
ck $G_{1:N}$' to: 'Depth S
tack ($D_{1:K}$)'",
      "from: 'Mask $M_2$'
to: 'Masked $I_2$'",
      "from: 'Mask $M_1$'
to: 'Masked $I_1$'",
      "from: 'Masked $I_2
$' to: 'Depth $D_2$'",
      "from: 'Masked $I_1
$' to: 'Projected $I_2$'",
      "from: 'Depth $D_2$'
to: 'Projected $I_2$'",
      "from: 'Projected $I
_2$' to: 'Inpaint Mask
$M'_1$'",
      "from: 'Inpaint Mask
$M'_1$' to: 'Small Mask
$M'_1$'",
      "from: 'Rendered
$I'_1$' to: 'Inpainted $I^
{In}_1$'",
      "from: 'Small Mask
$M'_1$' to: 'Inpainted $I^
{In}_1$'",
      "from: 'Depth $D^{I
n}_1$' to: 'Projection Box
$P_1$'",
      "from: 'Projection B
ox $P_1$' to: 'Projection
```

```
          {
            "name": "from:
'Image Stack ($I_{1:K}$)'
to: 'Gaussian Block $G_{1:
N}$'",
            "new_frequenc
y": 1
          }
        ]
      },
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "fram
e_2.png",
      "general_frame_reaso
ning": "The depth stack an
d the edge from the Gaussi
an model to it transition
to opaque.",
      "frequency_updates_t
his_frame": {
        "blocks": [],
        "nodes": [
          {
            "name": "Depth
Stack ($D_{1:K}$)",
```

```
Stack $I^P_{2:K}$'",
      "from: 'Projection B
ox $P_1$' to: 'Gaussian Bl
ock $G'_{1:N'}$",
      "from: 'Gaussian Blo
ck $G'_{1:N'}$' to: 'Rende
r Stack $I'_{2:K}$'",
      "from: 'Gaussian Blo
ck $G'_{1:N'}$' to: 'Rende
red $I'_1$'",
      "from: 'Projection S
tack $I^P_{2:K}$' to: 'Ren
der Stack $I'_{2:K}$'",
      "from: 'Depth $D^{I
n}_1$' to: 'Depth $D'_1$'"
    ]
  }
}
```

```
            "new_frequenc
y": 1
          }
        ],
        "edges": [
          {
            "name": "from:
'Gaussian Block $G_{1:N}$'
to: 'Depth Stack ($D_{1:K}
$)'",
            "new_frequenc
y": 1
          }
        ]
      },
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "fram
e_3.png",
      "general_frame_reaso
ning": "The view boxes and
their internal image/mask
components become opaqu
e.",
      "frequency_updates_t
his_frame": {
```

```
        "blocks": [
          {
            "name": "Box I
2, M2",
            "new_frequenc
y": 1
          },
          {
            "name": "Box I
1, M1",
            "new_frequenc
y": 1
          }
        ],
        "nodes": [
          {
            "name": "Image
$I_2$",
            "new_frequenc
y": 1
          },
          {
            "name": "Mask
$M_2$",
            "new_frequenc
y": 1
          },
          {
            "name": "Image
$I_1$",
            "new_frequenc
y": 1
          },
          {
            "name": "Mask
$M_1$",
```

```
            "new_frequenc
y": 1
          }
        ],
        "edges": []
      },
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "fram
e_4.png",
      "general_frame_reaso
ning": "Masked background
images and the edges from
the masks are revealed.",
      "frequency_updates_t
his_frame": {
        "blocks": [],
        "nodes": [
          {
            "name": "Maske
d $I_2$",
            "new_frequenc
y": 1
          },
          {
            "name": "Maske
```

```
d $I_1$",
            "new_frequenc
y": 1
          }
        ],
        "edges": [
          {
            "name": "from:
'Mask $M_2$' to: 'Masked
$I_2$'",
            "new_frequenc
y": 1
          },
          {
            "name": "from:
'Mask $M_1$' to: 'Masked
$I_1$'",
            "new_frequenc
y": 1
          }
        ]
      },
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "fram
e_5.png",
```

```
      "general_frame_reasoning": "Depth map $D_2$ and its connecting edge from Masked $I_2$ become opaque.",
      "frequency_updates_this_frame": {
        "blocks": [],
        "nodes": [
          {
            "name": "Depth $D_2$",
            "new_frequency": 1
          }
        ],
        "edges": [
          {
            "name": "from: 'Masked $I_2$' to: 'Depth $D_2$'",
            "new_frequency": 1
          }
        ]
      },
      "repetition_evaluation": {
        "justification_reasoning": "No repetitions to evaluate",
        "unjustified_freq_increase_element": [],
        "unjustified_freq_increase_arrow": []
      }
```

```
    },
    {
      "frame_label": "fram
e_6.png",
      "general_frame_reaso
ning": "Projected image $I
_2$ and the edges from $D_
2$ and Masked $I_1$ become
opaque.",
      "frequency_updates_t
his_frame": {
        "blocks": [],
        "nodes": [
          {
            "name": "Proje
cted $I_2$",
            "new_frequenc
y": 1
          }
        ],
        "edges": [
          {
            "name": "from:
'Depth $D_2$' to: 'Project
ed $I_2$'",
            "new_frequenc
y": 1
          },
          {
            "name": "from:
'Masked $I_1$' to: 'Projec
ted $I_2$'",
            "new_frequenc
y": 1
          }
        ]
```

```
      },
      "repetition_evaluation": {
        "justification_reasoning": "No repetitions to evaluate",
        "unjustified_freq_increase_element": [],
        "unjustified_freq_increase_arrow": []
      }
    },
    {
      "frame_label": "frame_7.png",
      "general_frame_reasoning": "The inferred inpaint mask $M'_1$ and the edge from Projected $I_2$ transition to opaque.",
      "frequency_updates_this_frame": {
        "blocks": [],
        "nodes": [
          {
            "name": "Inpaint Mask $M'_1$",
            "new_frequency": 1
          }
        ],
        "edges": [
          {
            "name": "from: 'Projected $I_2$' to: 'Inpaint Mask $M'_1$'",
```

```
            "new_frequenc
y": 1
          }
        ]
      },
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "fram
e_8.png",
      "general_frame_reaso
ning": "The small version
of the inpaint mask and th
e dashed edge from the lar
ger mask become opaque.",
      "frequency_updates_t
his_frame": {
        "blocks": [],
        "nodes": [
          {
            "name": "Small
Mask $M'_1$",
            "new_frequenc
y": 1
          }
        ],
        "edges": [
```

```
          {
            "name": "from:
'Inpaint Mask $M'_1$' to:
'Small Mask $M'_1$'",
            "new_frequenc
y": 1
          }
        ]
      },
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "fram
e_9.png",
      "general_frame_reaso
ning": "The composite inpa
inting block (including $D
^{In}_1$ and $I^{In}_1$) a
nd the edge from the small
mask become opaque.",
      "frequency_updates_t
his_frame": {
        "blocks": [],
        "nodes": [
          {
            "name": "Compo
site Depth/Image $D^{In}_
```

```
1, I^{In}_1$",
            "new_frequenc
y": 1
          },
          {
            "name": "Depth
$D^{In}_1$",
            "new_frequenc
y": 1
          },
          {
            "name": "Inpai
nted $I^{In}_1$",
            "new_frequenc
y": 1
          }
        ],
        "edges": [
          {
            "name": "from:
'Small Mask $M'_1$' to: 'I
npainted $I^{In}_1$'",
            "new_frequenc
y": 1
          }
        ]
      },
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
```

```
      }
    }
  ],
  "final_unjustified_repet
itions": {
    "elements": [],
    "arrows": []
  },
  "unnecessary_repetition_
count_element": 0,
  "unnecessary_repetition_
count_arrow": 0
}
```

Hopping Bounding Box

```
You are an expert Diagram Animation Repetition Detector. Your
objective is to track the frequency of appearances of element
s from a precomputed checklist and verify if any elements are
erroneously re-highlighted in the animation sequence.

You are provided with:
1. The Original Diagram Image.
2. A sequence of Animation Frames in chronological order.
3. The Initial Animation Checklist containing "blocks", "node
s", and "edges".

*** GENERIC INSTRUCTIONS ***
1. Frequency Counter: Treat the checklist as a frequency coun
ter where every block and node starts at `0`.
2. The State Change Rule (CRITICAL): An element's frequency i
ncreases by +1 ONLY when the bounding box *newly arrives* on
```

```
it. If the box rests on the same element for multiple frames, DO NOT increase the frequency again.
3. Repetition Justification: If an element's frequency reaches 2 or more (meaning the box left and came back), you MUST reason if it is justified before flagging it. Justified repetitions include returning to a central hub node, or a parent container re-highlighted after scanning its children. If unjustified, flag it.
4. Semantic Mapping: Use the Original Image to logically map descriptive names in the checklist to visual shapes.

*** STYLE ADAPTER: HOPPING BOUNDING BOX ***
In Hopping Bounding Box, a transparent highlight box discretely jumps from one element to the next.
- State Change Trigger: Increase frequency (+1) the exact moment the bounding box lands on and encloses an element.
- SPECIAL EDGE EXCEPTION: Bounding boxes DO NOT highlight arrows. You must COMPLETELY IGNORE the "edges" list. Never update edge frequencies.
- You must always output `0` for `unnecessary_repetition_count_arrow` and empty lists for arrow tracking.

*** OUTPUT REQUIREMENT ***
Output ONLY valid JSON matching the exact schema below.

```json
{
  "metadata": {
    "animation_style": "Hopping Bounding Box"
  },
  "timestep_reasoning": [
    {
      "frame_label": "string - frame filename or index",
      "general_frame_reasoning": "Detailed rationale of what the box newly landed on this frame based on the state change rule.",
```
```

```
      "frequency_updates_this_frame": {
        "blocks": [{"name": "Block Name", "new_frequency":
1}],
        "nodes": [{"name": "Node Name", "new_frequency": 1}]
      },
      "repetition_evaluation": {
        "justification_reasoning": "Deeply reason about ANY e
lement whose frequency is now >= 2. Is returning to this node
a logical traversal choice (e.g., returning to a hub/parent)?
Or a glitch? If no elements reached frequency >= 2, state 'No
repetitions to evaluate'.",
        "unjustified_freq_increase_element": ["List of block/
node names whose unjustified frequency increased this fram
e"],
        "unjustified_freq_increase_arrow": []
      }
    }
  ],
  "final_unjustified_repetitions": {
    "elements": ["Cumulative list of all unjustified repeated
blocks/nodes"],
    "arrows": []
  },
  "unnecessary_repetition_count_element": 0,
  "unnecessary_repetition_count_arrow": 0
}
```


AnimateBanana Animation Video (TikZ)

attachment:7c2c7abb-446d-4155-9631-829057f92872:CVPR_2025_pipe00011_video_tikz_HB.mp4


```
{
  "metadata": {
    "animation_style": "Ho
pping Bounding Box"
  },
  "timestep_reasoning": [
    {
      "frame_label": "Fram
```

Full pass can be seen at

https://chat.qwen.ai/s/b421a461-3d4f-4612-940c-ebe01ec12799?fev=0.2.84


```
{
  "animation_checklist": {
    "blocks": [
      "Phase 1 & 2 Container"
    ],
    "nodes": [
      "Initializing Process",
      "Pre-trained 3D-GS $\mathcal{G}_o$",
      "Frequency Guided Densification (FGD)",
      "Pre-trained 3D-GS $\mathcal{G}_o$ (Phase 1)",
      "Remove 3D Gaussians",
      "Ship Image (Phase 2)",
      "Patched Image (Phase 2)",
      "Fourier Transform Discrete",
      "Frequency Grid (Phase 2)",
      "High Frequency Detection Mask",
      e 30",
      "general_frame_reasoning": "The bounding box newly arrives on '$\\mathcal{L}_{rec}, \\mathcal{L}_{lpips}$', highlighting the reconstruction and perceptual loss components for the first time as the animation explains the visual fidelity preservation mechanism.",
      "frequency_updates_this_frame": [
        {
          "name": "$\\mathcal{L}_{rec}, \\mathcal{L}_{lpips}$",
          "new_frequency": 1
        }
      ],
      "repetition_evaluation": {
        "justification_reasoning": "No repetitions to evaluate",
        "unjustified_freq_increase_element": [],
        "unjustified_freq_increase_arrow": []
      }
    },
    {
      "frame_label": "Frame 31",
```

```
      "Select High Frequen
cy",
      "Split 3D Gaussian
s",
      "After FGD 3D-GS $\m
athcal{G}_o'$",
      "Gradient Mask",
      "Hadamard Product
($\odot$)",
      "Optimizer",
      "Fine-tuned 3D-GS
$\mathcal{G}_w$",
      "Fine-tuning Proces
s",
      "3D Render (Ship)",
      "Transform Wavelet D
iscrete",
      "Low Frequency",
      "High Frequency",
      "Pre-trained Decode
r",
      "Decoded Message",
      "Fixed Message",
      "$\mathcal{L}_m$",
      "Ground Truth",
      "Transform Wavelet D
iscrete (GT)",
      "High Frequency (G
T)",
      "$\mathcal{L}_{rec},
\mathcal{L}_{lpips}$",
      "$\mathcal{L}_w$",
      "$\mathcal{L}_{tota
l}$"
    ],
    "edges": []
```

```
      "general_frame_reaso
ning": "The bounding box n
ewly arrives on '$\\mathca
l{L}_w$', highlighting the
wavelet subband loss compo
nent for the first time as
the animation details the
fine detail preservation m
echanism.",
      "frequency_updates_t
his_frame": [
        {
          "name": "$\\math
cal{L}_w$",
          "new_frequency":
1
        }
      ],
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "Fram
e 32",
      "general_frame_reaso
ning": "The bounding box r
eturns to 'High Frequency'
after previously highlight
```

```
  }
}
```

```
ing it in Frame 24. This i
s the second arrival on th
is node.",
      "frequency_updates_t
his_frame": [
        {
          "name": "High Fr
equency",
          "new_frequency":
2
        }
      ],
      "repetition_evaluati
on": {
        "justification_rea
soning": "The 'High Freque
ncy' node represents a cor
e component that appears i
n both the main image proc
essing path and the ground
truth processing path. The
animation first explains t
he high-frequency componen
t in the main image contex
t (Frame 24), then returns
to highlight it in the gro
und truth context. This is
a justified repetition as
it demonstrates parallel p
rocessing of the same conc
eptual component across tw
o different data streams
(watermarked image vs grou
nd truth).",
        "unjustified_freq_
increase_element": [],
```

```
        "unjustified_freq_
increase_arrow": []
      }
    },
    {
      "frame_label": "Fram
e 33",
      "general_frame_reaso
ning": "The bounding box n
ewly arrives on '$\\mathca
l{L}_{total}$', highlighti
ng the total loss componen
t for the first time as th
e animation explains how i
ndividual losses are combi
ned to drive optimizatio
n.",
      "frequency_updates_t
his_frame": [
        {
          "name": "$\\math
cal{L}_{total}$",
          "new_frequency":
1
        }
      ],
      "repetition_evaluati
on": {
        "justification_rea
soning": "No repetitions t
o evaluate",
        "unjustified_freq_
increase_element": [],
        "unjustified_freq_
increase_arrow": []
      }
```

```
    }
  ],
  "final_unjustified_repetitions": {
    "elements": [],
    "arrows": []
  },
  "unnecessary_repetition_count_element": 0,
  "unnecessary_repetition_count_arrow": 0
}
```

Sliding Bounding Box

```
You are an expert Diagram Animation Repetition Detector. Your objective is to track the frequency of appearances of elements from a precomputed checklist and verify if any elements are erroneously re-highlighted in the animation sequence.

You are provided with:
1. The Original Diagram Image.
2. A sequence of Animation Frames in chronological order.
3. The Initial Animation Checklist containing "blocks", "nodes", and "edges".

*** GENERIC INSTRUCTIONS ***
1. Frequency Counter: Treat the checklist as a frequency counter where every block and node starts at `0`.
2. The State Change Rule (CRITICAL): An element's frequency increases by +1 ONLY when the bounding box *comes to rest* on it. If the box rests on the same element for multiple frames, DO NOT increase the frequency again. Ignore the box while it
```

```
is mid-slide.
3. Repetition Justification: If an element's frequency reache
s 2 or more (meaning the box left and came back), you MUST re
ason if it is justified before flagging it. Justified repetit
ions include returning to a central hub node, or a parent con
tainer re-highlighted after scanning its children. If unjusti
fied, flag it.
4. Semantic Mapping: Use the Original Image to logically map
descriptive names in the checklist to visual shapes.

*** STYLE ADAPTER: SLIDING BOUNDING BOX ***
In Sliding Bounding Box, a transparent highlight box smoothly
slides across the canvas.
- State Change Trigger: Increase frequency (+1) the exact mom
ent the bounding box finishes its slide and comes to rest enc
losing an element.
- SPECIAL EDGE EXCEPTION: Bounding boxes DO NOT highlight arr
ows. You must COMPLETELY IGNORE the "edges" list. Never updat
e edge frequencies.
- You must always output `0` for `unnecessary_repetition_coun
t_arrow` and empty lists for arrow tracking.

*** OUTPUT REQUIREMENT ***
Output ONLY valid JSON matching the exact schema below.

```json
{
  "metadata": {
    "animation_style": "Sliding Bounding Box"
  },
  "timestep_reasoning": [
    {
      "frame_label": "string - frame filename or index",
      "general_frame_reasoning": "Detailed rationale of what
the box newly came to rest on this frame based on the state c
hange rule.",
```
```

```
      "frequency_updates_this_frame": {
        "blocks": [{"name": "Block Name", "new_frequency":
1}],
        "nodes": [{"name": "Node Name", "new_frequency": 1}]
      },
      "repetition_evaluation": {
        "justification_reasoning": "Deeply reason about ANY e
lement whose frequency is now >= 2. Is returning to this node
a logical traversal choice (e.g., returning to a hub/parent)?
Or a glitch? If no elements reached frequency >= 2, state 'No
repetitions to evaluate'.",
        "unjustified_freq_increase_element": ["List of block/
node names whose unjustified frequency increased this fram
e"],
        "unjustified_freq_increase_arrow": []
      }
    }
  ],
  "final_unjustified_repetitions": {
    "elements": ["Cumulative list of all unjustified repeated
blocks/nodes"],
    "arrows": []
  },
  "unnecessary_repetition_count_element": 0,
  "unnecessary_repetition_count_arrow": 0
}
```


Note: Sliding bounding box will be pretty similar to Hopping bounding box, with just the difference of more frames involved in the slide. Since the animations at this stage have already passed the style check, we know for sure that the sliding bounding box is working really well. As a result, sample after every 4 frames in TikZ to save on the input frames per sample